PROYECTO FINAL

Sistema de Comercio Electrónico en Java

Programación Orientada a Objetos — Java POO

Descripción general

Cada grupo diseñará, desarrollará y presentará un sistema de comercio electrónico funcional en Java, aplicando los principios de la Programación Orientada a Objetos. El proyecto con mayor votación dentro de cada curso competirá en una etapa final entre los cuatro cursos.

Modalidad Trabajo en grupo (máx. 4 integrantes)	**Entrega** Repositorio público en GitHub
Lenguaje base Java (frameworks compatibles permitidos)	**Selección de representante** Votación entre grupos del mismo curso

1. Objetivo

Que cada grupo desarrolle un sistema de comercio electrónico completo en Java, demostrando la correcta aplicación de los cuatro pilares de la Programación Orientada a Objetos —herencia, encapsulación, polimorfismo y abstracción—, así como el uso de colecciones, manejo de excepciones e interfaces gráficas o web funcionales con persistencia de datos.

2. Descripción de la actividad

La actividad se desarrolla en dos etapas sucesivas:

Etapa 1 — Desarrollo del proyecto por grupo

Cada grupo construye su propio sistema de ecommerce en Java. El proyecto debe ser funcional, estar publicado en un repositorio público de GitHub y cumplir con los requisitos técnicos y de documentación descritos en este documento.

Etapa 2 — Votación y selección del representante

Una vez publicados todos los proyectos del curso, los grupos votan para elegir cuál proyecto los representará en la etapa final. El ganador de cada curso compite contra los representantes de los otros tres cursos en una votación final.

3. Requisitos técnicos del sistema

3.1 Funcionalidades mínimas obligatorias

El sistema debe implementar, como mínimo, las siguientes funcionalidades:

- Gestión de productos: creación, consulta, modificación y eliminación, con control de inventario (stock).
- Gestión de usuarios: registro e inicio de sesión, distinguiendo al menos entre el rol de cliente y el rol de administrador.
- Carrito de compras: agregar y eliminar productos, modificar cantidades y calcular totales en tiempo real.
- Flujo completo de pedido: desde la confirmación del carrito, selección del método de pago, hasta la generación del pedido y la actualización del inventario.
- Historial de pedidos: el cliente puede consultar sus pedidos anteriores con estado y detalle.
- Interfaz funcional: interfaz gráfica de escritorio o aplicación web operativa. No se acepta como entrega única una aplicación de consola.
- Persistencia de datos: los datos deben sobrevivir al cierre de la aplicación (archivos, base de datos u otro mecanismo).

3.2 Conceptos de POO obligatorios

El código del proyecto debe evidenciar de forma explícita y funcional los siguientes conceptos:

- Herencia: al menos una jerarquía de clases con relación padre–hijo significativa dentro del dominio del sistema (por ejemplo, Usuario → Cliente / Administrador, o Producto → subtipos).
- Encapsulación: todos los atributos de las entidades del modelo deben declararse como privados, con acceso controlado mediante métodos getter y setter.
- Polimorfismo: al menos un método sobrescrito con @Override que participe activamente en un flujo funcional del sistema.
- Abstracción: uso de al menos una clase abstracta o interfaz que modele un concepto general del dominio (por ejemplo, una clase abstracta Pago con subclases para distintos métodos de pago).
- Colecciones: uso de ArrayList u otra colección Java para gestionar listas de entidades (productos, pedidos, usuarios, etc.).
- Manejo de excepciones: uso de bloques try/catch en operaciones que puedan fallar (stock insuficiente, datos de entrada inválidos, errores de conexión a BD, etc.).

3.3 Tecnología permitida

No existe restricción sobre las tecnologías asociadas al ecosistema Java. Los grupos pueden elegir libremente su stack tecnológico dentro de las siguientes categorías:

Categoría	Opciones permitidas
Interfaz de usuario	Swing, JavaFX, Spring Boot + Thymeleaf, Spring Boot + API REST con frontend propio
Persistencia	Archivos (.dat, .txt, .csv), SQLite, MySQL, PostgreSQL, H2, JPA/Hibernate
Build / Gestión de dependencias	Sin build tool, Maven, Gradle
IDE	NetBeans, IntelliJ IDEA, Eclipse, VS Code con extensiones Java
Control de versiones	Git + GitHub (repositorio público obligatorio)

Nota: el uso de Spring Boot u otros frameworks está permitido siempre que las entidades del modelo de dominio estén implementadas en Java con POO explícita. No se acepta como modelo de dominio un conjunto de clases compuestas únicamente por anotaciones de framework sin jerarquía, abstracción ni polimorfismo reales.

4. Entregables

Cada grupo debe publicar en su repositorio de GitHub los siguientes entregables antes de la fecha límite indicada por el docente:

Entregable	Descripción	Obligatorio
Repositorio GitHub público	Código fuente completo, historial de commits visible, repositorio accesible.	Sí
README.md	Descripción del proyecto, integrantes, instrucciones de ejecución y tecnologías usadas.	Sí
Diagrama de clases UML	Exportado como imagen PNG en la carpeta docs/uml/ del repositorio.	Sí
Diagrama de casos de uso	Cubre los flujos principales del sistema. Incluido en docs/uml/.	Sí
Script SQL (si aplica)	Script de creación de base de datos en db/schema.sql.	Si usan BD

El repositorio debe ser público y estar accesible sin autenticación. Un integrante del grupo comparte el enlace en el grupo de WhatsApp del curso con el siguiente formato:

```
👉 Grupo [N] — [Nombre del proyecto] —
https://github.com/usuario/repo
```

Repositorio base del proyecto:
`https://github.com/Antomaker/competencia.git` — *Clona este repositorio como punto de partida y construye tu proyecto sobre él.*

5. Proceso de votación

5.1 Votación interna por curso

Una vez publicados todos los repositorios del curso, el docente abre la votación para elegir el proyecto representante. Las reglas son las siguientes:

Aspecto	Detalle
Votos por grupo	Cada grupo dispone de 3 votos para distribuir.
Voto propio	Exactamente 1 voto debe destinarse al proyecto del propio grupo. Es obligatorio y no puede omitirse.
Votos externos	Los 2 votos restantes pueden destinarse al mismo proyecto externo o a dos proyectos distintos. No se permite votar dos veces por el propio grupo.
Desempate	En caso de empate en votos totales, se aplica como criterio de desempate la cantidad de votos externos recibidos. Si el empate persiste, el docente decide.
Representante	El proyecto con mayor puntuación representa al curso en la etapa final entre los cuatro cursos.

5.2 Etapa final entre cursos

Los cuatro proyectos ganadores —uno por cada curso— compiten entre sí aplicando exactamente las mismas reglas de votación. El resultado de esta etapa determina el mejor proyecto de ecommerce Java del semestre.

6. Restricciones y condiciones

- El repositorio debe contener únicamente código fuente y recursos necesarios para ejecutar el proyecto. No se deben incluir archivos compilados (.class, .jar), credenciales de bases de datos ni carpetas privadas generadas por el IDE.
- Todos los integrantes del grupo deben figurar en el README con su nombre completo y usuario de GitHub.
- El historial de commits del repositorio debe reflejar trabajo continuo del equipo. Un único commit con todo el proyecto no es aceptable.
- El proyecto debe ejecutarse correctamente siguiendo únicamente las instrucciones del README. Si no puede ejecutarse, no participa en la votación.
- Queda prohibido el uso de código generado íntegramente por herramientas de inteligencia artificial sin comprensión ni adaptación por parte del grupo. El docente puede realizar preguntas sobre cualquier parte del código durante la presentación.

7. Recomendaciones

Se sugiere tener en cuenta lo siguiente durante el desarrollo:

- Iniciar el repositorio desde el primer día de trabajo y realizar commits con mensajes descriptivos al finalizar cada sesión.
- Distribuir responsabilidades claras entre los integrantes desde el inicio: modelo de dominio, capa de datos, interfaz de usuario y documentación.
- Diseñar el diagrama UML de clases antes de escribir código. Un diagrama bien pensado evita rediseños costosos en etapas avanzadas.
- Probar que el proyecto se ejecuta correctamente en un computador distinto al del desarrollo, siguiendo solo las instrucciones del README, antes de compartir el enlace.
- Si se utiliza una base de datos externa (MySQL, PostgreSQL), incluir el script de creación y verificar que las instrucciones de configuración sean claras para un usuario nuevo.

Documento preparado para el curso Java POO — Programación Orientada a Objetos
Cualquier duda o aclaración debe dirigirse al docente del curso.